

Session 07 — Study Guide: From Express to NestJS

Status: PREPARED DRAFT

Where the session ends

By the end of this session, B4F Board — the main project of this course — has a real, running NestJS backend (`api/`) with two complete features: Projects (built live, by the instructor) and Workspaces (built by your team), each with its own module, controller, and service, wired together through dependency injection. There is still no database — everything lives in memory, exactly like B4F Hub's own `server/store.js` — and no authentication yet. This is the real start of B4F Board's backend, not a throwaway exercise.

Why a new project, not more B4F Hub?

B4F Hub (the bridge project) is finished as of Session 6 — it taught Node/Express fundamentals using a real, already-existing server. B4F Board is a separate, bigger project: a Jira-style project-tracking tool, built from here through the rest of the course, eventually with real authentication and a real database. Today starts its backend, in its own repository. Nothing from B4F Hub's own code is reused — only the *concepts* you already learned (routing, controllers, shared state, validation, status codes) carry forward.

Node, Express, and Nest — what each one actually is

Node.js is the JavaScript runtime — the program that actually executes your code outside a browser, handles the network socket, and runs the event loop. Nothing about this changes today.

Express is a library that runs inside Node and gives you `app.get()`, `app.post()`, middleware, and `req / res` objects — a thin, flexible toolkit with very few opinions about how you organize your own files.

NestJS is a framework that also runs inside Node, and by default runs *on top of* Express (`NestFactory.create()` builds a real Express application internally — you can even access it directly if you ever needed to). Nest adds a required structure (modules, controllers, providers) and a dependency-injection container on top of what Express already gives you. **Nest does not replace Node, and it does not replace Express — it organizes what they already do.**

A request to a Nest app still travels exactly the same path it always has: TCP socket → Node's HTTP parsing → (by default) Express's own request handling → whichever piece of code is registered to handle that path and method. Nest decides *which code* runs through decorators and modules instead of through `router.get(path, handler)` calls you write by hand — but underneath, for a default Nest app, it is still Express doing the actual routing.

Why learn Express before Nest?

Because every concept Nest introduces today is a *name* for something you already built by hand in Sessions 5-6: a router file, a controller file, a shared state module, manual `import / mount` wiring. Learning Nest first would mean learning its rules without knowing what problem they solve. Learning Express first

means every new Nest word this session has a concrete, already-understood target to attach to.

Bootstrap: `main.ts`

TS

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

`NestFactory.create(AppModule)` builds the whole application from one root description (the root module) and returns it. `app.listen(...)` is the exact same idea as Express's own `app.listen(...)` — because, underneath, it is calling it.

Modules: `@Module()`

A module is a *grouping* — it tells Nest which controllers and providers (services) belong together, and which other modules it needs. The root module (`AppModule`) is the one Nest starts from; every feature module gets pulled in through its `imports` array.

TS

```
@Module({
  imports: [ProjectsModule, WorkspacesModule],
  controllers: [],
  providers: [],
})
export class AppModule {}
```

A module on its own does nothing until something that Nest already knows about imports it — tracing all the way back to the root module. This is exactly why, in class, creating `ProjectsModule` did not make any route live until it was added to `AppModule`'s `imports` array.

Controllers: `@Controller()`, `@Get()`, `@Param()`

A controller's job is identical to an Express router file: decide which incoming request reaches which method.

TS

```
@Controller('projects')
export class ProjectsController {
  constructor(private readonly projectsService: ProjectsService) {}

  @Get()
  findAll() {
    return this.projectsService.findAll();
  }

  @Get('/:id')
  findOne(@Param('id') id: string) {
    return this.projectsService.findOne(Number(id));
  }
}
```

`@Controller('projects')` sets the path prefix — every route in this class starts with `/projects`. `@Get()` with no argument matches the bare prefix; `@Get('/:id')` adds a dynamic segment, read with `@Param('id')`. Whatever a method *returns* (a plain object or array here) is automatically turned into a JSON response — you never call `res.json(...)` yourself.

Decorators, practically

A decorator (`@Something`) attaches metadata to the class, method, or parameter right below/after it — it tells Nest "here is this thing's role." You do not need to know how decorators work internally this session; the mental model "this label tells Nest what this is for" is enough for everything taught today: `@Module`, `@Controller`, `@Get`, `@Patch`, `@Param`, `@Body`, `@Injectable`.

Services and providers: `@Injectable()`

A service holds the actual logic and data a feature needs — the equivalent of our Express controller functions plus `store.js`'s shared array, combined into one class.

TS

```
@Injectable()
export class ProjectsService {
  private readonly projects: Project[] = seedProjects;

  findAll(): Project[] {
    return this.projects;
  }
  // ...
}
```

`@Injectable()` marks a class as something Nest's container is allowed to construct and hand to other classes — which is exactly what makes dependency injection possible.

Dependency injection, at the simplest scale

TS

```
constructor(private readonly projectsService: ProjectsService) {}
```

The controller never writes `new ProjectsService()`. It just declares, in its constructor, that it needs one — and Nest's container constructs it and hands it over automatically, because the service's own module listed it in `providers`. This matters because it means a class only has to know *what* it needs, never *how* to build it — the same instinct behind why our Express controllers never built their own copy of `store.js`'s array, just imported the one shared instance.

A real mutation: `@Patch()` and `@Body()`

Reads are not the whole story — B4F Board's `Project` has a real unique `key` field (used later for issue keys like `WEB-42`), so updating a project's `key` has a genuine conflict to guard against:

TS

```
@Patch('/:id')
update(
  @Param('id') id: string,
  @Body() body: { name?: string; key?: string },
) {
  return this.projectsService.update(Number(id), body);
}
```

`@Body()` gives you the already-parsed request body directly, the same role `req.body` played in Express. The service's own `update()` method validates the input (400 for an empty name), checks the real conflict (409 if the new key is already used by another project), then applies the change — in that order, because checking a conflict before confirming the project even exists would be backwards.

The request lifecycle, end to end

1. A request arrives at the Node process (same as always).
2. Nest's underlying Express instance matches it to a registered route.
3. Nest finds the controller method for that route and resolves its constructor dependencies (the services it needs) from its container.
4. The method runs, usually delegating the actual work to a service.
5. Whatever the method returns is serialized to JSON automatically.
6. If the method (or anything it calls) throws a Nest exception (like `NotFoundException`), Nest catches it and produces the matching HTTP status + JSON error body — you never write that conversion yourself.

Express → Nest mapping table

Express	Nest
Node runtime	still underneath, unchanged
<code>server/index.js</code> , <code>app.listen()</code>	<code>main.ts</code> , <code>NestFactory.create()</code> + <code>app.listen()</code>
<code>const app = express()</code>	the Nest application Nest builds for you
<code>express.Router()</code>	a controller's own routing
<code>router.get(path, fn)</code> / <code>.post</code> / <code>.patch</code>	<code>@Get()</code> / <code>@Post()</code> / <code>@Patch()</code>
<code>req.params.id</code>	<code>@Param('id')</code>
<code>req.body</code>	<code>@Body()</code>
a controller function	a controller method
shared logic in <code>controllers/*.js</code> + <code>store.js</code>	a service/provider (<code>@Injectable()</code>)
manual <code>import</code> + <code>app.use(...)</code> wiring	module registration + dependency injection
a feature's router+controller+shared state	a feature module

Common beginner mistakes

- **Writing `new SomeService()` inside a controller.** This defeats the entire point of DI, and breaks the moment that service itself needs something injected. Always declare it as a constructor parameter instead.
- **Forgetting to register a module in `AppModule`'s `imports`.** Every file can exist and compile correctly while still being completely unreachable — a module only "counts" once something Nest already knows about imports it.
- **Confusing `@Param()` with `req.params`.** `@Param('id')` already gives you just that one value (as a string) — you do not read it off a `params` object the way Express does.
- **Putting business logic directly in a controller method.** The controller's job is routing and delegation only; real logic belongs in the service, exactly like our Express controllers never put real logic directly in `index.js`.
- **Checking for a conflict before confirming the resource exists.** In `update()`, a missing project should 404 before anything else is checked — getting this order backwards can mean reading a field off something that doesn't exist.
- **Assuming Nest "auto-validates" anything.** Nest does not inspect your `@Body()` payload's shape unless you add a validation mechanism — this session's homework validates manually with plain `if` statements, exactly like our Express controllers already do.

Where databases fit later

Nothing in B4F Board's `api/` touches a database yet — all data is a small, in-memory array, created when the service class is constructed, lost when the server restarts, exactly like every B4F Hub session before it. A real database (MongoDB first, then PostgreSQL) is introduced in a later stage of the course, once this

backend's foundation is further along.

Not yet

These exist in NestJS, but are deliberately **not** covered this session — do not use them yet, and do not worry if you have seen the names before:

- DTOs and `class-validator` decorators (`@IsString()`, `@IsNotEmpty()`, etc.)
- `ValidationPipe` or any pipe
- Guards, interceptors, custom exception filters
- Provider tokens, custom providers (`useClass` / `useValue` / `useFactory`), injection scopes
- Dynamic modules, global modules
- Swagger/OpenAPI
- Any database (MongoDB, PostgreSQL, or otherwise)
- Authentication of any kind

Glossary

- **Decorator** — `@Something`, attaches metadata to the class/method/parameter right after it.
- **Module** — a class decorated with `@Module()`, grouping a feature's controllers/providers.
- **Controller** — a class decorated with `@Controller()`, routing requests to methods.
- **Service / provider** — a class decorated with `@Injectable()`, holding logic/data; a service is the most common kind of provider.
- **Dependency injection (DI)** — supplying a class's dependencies via its constructor instead of it constructing them itself.
- **Bootstrap** — building and starting the app, done once, in `main.ts`.

Review questions

1. What does `NestFactory.create(AppModule)` actually do?
2. Why does a brand-new feature module do nothing on its own, without one more step?
3. What is the Express equivalent of `@Get('/:id')`? Of `@Param('id')`? Of `@Body()`?
4. In `ProjectsService.update()`, why does the conflict check happen *after* `findOne`, not before?

Answers

1. It reads the whole module graph starting from the root module, builds and connects every provider/controller, registers every route with the underlying HTTP adapter, and returns the running application.
2. Because nothing imports it yet — a module only becomes reachable once something Nest already knows about (tracing back to the root module) lists it in an `imports` array.
3. `router.get('/:id', handler); req.params.id; req.body`.

4. Because a missing project should produce a 404, not a 409 — checking the conflict first could mean comparing a field on a project that was never found in the first place.

What you should be able to explain now

- What NestJS is, and what it is not, in one sentence each.
- Why B4F Board starts as a separate project from B4F Hub, not a continuation of it.
- Why a feature module does nothing until it is imported somewhere Nest already knows about.
- The full Express → Nest mapping table, from memory.
- Why `new SomeService()` inside a controller defeats the point of dependency injection.